Proyecto 2 DANN-ALPES

Fundamentos de Bases de Datos

Universidad de los Andes

Estudiantes:
Martin Chicaiza – 202420350
Daniela Martinez – 202310127
Santiago Rojas – 202221434

1. Introducción

Dann-Alpes es una cadena hotelera que requiere un sistema de información para gestionar hoteles, habitaciones, servicios adicionales, clientes y reservas. En la primera entrega construimos un modelo conceptual y relacional del sistema. En esta segunda entrega se implementó dicho modelo en una base de datos mediante Apex Oracle, realizamos la población de los datos y consultas SQL.

2. Análisis y revisión del modelo de datos

Para esta entrega se revisó el modelo construido en la primera entrega y se ajustó de acuerdo con la retroalimentación recibida, principalmente la obligatoriedad de algunos atributos.

En la primera entrega ya se habían aplicado correctamente las formas normales, incluyendo la Forma Normal de Boyce-Codd (BCNF), a todas las relaciones del modelo. Tras la revisión, se confirmó que cada relación cumple con BCNF, por lo que los cambios realizados en esta entrega fueron mínimos: se ajustaron únicamente las restricciones de obligatoriedad (NOT NULL) en ciertos atributos según la retroalimentación, sin necesidad de reestructurar ninguna tabla.

3. Implementación del esquema de la base de datos

La implementación del esquema se basó directamente en las tablas del modelo relacional documentadas en la plantilla Excel, las cuales fueron derivadas del modelo Entidad-Relación (E/R) diseñado en la primera entrega. Se utilizó SQL para la creación de todas las tablas en Oracle APEX a través de SQL Workshop, incluyendo:

- 12 tablas con sus respectivas columnas y tipos de datos.
- Claves primarias (PK) con IDENTITY para generación automática de IDs.
- Claves foráneas (FK) con ON DELETE CASCADE donde corresponde.
- Constraints UNIQUE para evitar duplicados (ciudades, documentos, correos, códigos de confirmación).
- Constraints CHECK para validar costos, estados, tipos de cama, tipos de vista, entre otros.
- Triggers para validar que solo clientes puedan hacer reservas y para evitar solapamiento de fechas.
- Índices sobre columnas de búsqueda frecuente para optimizar el rendimiento.

El script SQL completo (schema_dann_alpes.sql) se encuentra anexo en el repositorio.

4. Población de tablas de la base de datos

La población de la base de datos fue un proceso extenso que requirió la generación de un volumen significativo de datos coherentes entre sí. Se utilizó un script SQL (seed_dann_alpes_completo.sql) para insertar todos los registros, cumpliendo con los requisitos mínimos establecidos:

- 15 hoteles diferentes distribuidos en 10 ciudades colombianas.
- 303 clientes inscritos (superando el mínimo de 300).
- 304 reservas con un promedio de 20.3 clientes por hotel.
- 52 habitaciones con camas, comodidades y servicios asociados.
- 11 servicios adicionales (los 6 de costo fijo y 5 de costo por consumo definidos en el enunciado).
- 5 administradores del sistema.

El proceso fue largo y requirió cuidado para mantener la consistencia referencial entre tablas, respetar los constraints CHECK y evitar solapamiento de reservas activas para una misma habitación.

5. Diseño de sentencias SQL

A continuación se presentan las sentencias SQL diseñadas para los requerimientos funcionales de consulta (RFC1 a RFC5). Para cada consulta se muestra primero la versión inicial y luego la versión corregida con la explicación de los cambios realizados.

5.1. RFC1 – Habitaciones disponibles por intervalo de fechas

Tablas utilizadas: HABITACIONES, HOTELES, RESERVAS. Se realiza un INNER JOIN entre HABITACIONES y HOTELES para obtener el nombre del hotel, y un subquery con NOT IN sobre RESERVAS para excluir habitaciones que ya tienen reservas activas en el rango de fechas.

Versión inicial:

```
SELECT habitaciones.HabitacionID
FROM habitaciones
INNER JOIN hoteles ON hoteles.hotelid = habitaciones.hotelid
WHERE habitaciones.habitacionid NOT IN (
    SELECT habitaciones.habitacionid
    FROM habitaciones
    INNER JOIN reservas ON reservas.habitacionasociada = habitaciones.habitacionid
    WHERE reservas.fechafin >= DATE '2026-04-06'
      AND reservas.fechainicio <= DATE '2026-04-05')
AND habitaciones.disponible = 1;
```

Versión corregida:

```
SELECT habitaciones.HabitacionID, habitaciones.Tipo,
       habitaciones.CostoNocheAlta, habitaciones.CostoNocheBaja,
       habitaciones.Dimensiones, habitaciones.TipoVista,
       hoteles.Nombre AS Hotel
FROM habitaciones
```

```
INNER JOIN hoteles ON hoteles.HotelID = habitaciones.HotelID
WHERE habitaciones.HabitacionID NOT IN (
    SELECT reservas.HabitacionAsociada
    FROM reservas
    WHERE reservas.Estado = 'activa'
      AND reservas.FechaFin >= :P_FECHA_INICIO
      AND reservas.FechaInicio <= :P_FECHA_FIN)
AND habitaciones.Disponible = 1
AND (:P_HOTEL IS NULL OR habitaciones.HotelID = :P_HOTEL);
```

Cambios realizados:

1. Se agregó filtro por hotel específico con parámetro opcional :P_HOTEL. 2. Se agregó filtro de solo reservas activas (Estado = 'activa') para que las canceladas no bloqueen habitaciones. 3. Se agregaron columnas descriptivas (Tipo, Costos, Dimensiones, TipoVista, nombre del Hotel) en vez de solo el HabitacionID. 4. Se parametrizaron las fechas con :P_FECHA_INICIO y :P_FECHA_FIN para uso en APEX. 5. Se corrigió el subquery para usar reservas.HabitacionAsociada directamente.

5.2. RFC2 – Reservas a través del tiempo

Tablas utilizadas: RESERVAS, HABITACIONES, HOTELES, CIUDADES. Se realizan INNER JOINs encadenados para conectar las reservas con el hotel y la ciudad. Se agrupa por período mensual (MM/YYYY) usando TO_CHAR.

Versión inicial:

```
SELECT COUNT(reservas.reservaid) AS num_reservas,
       hoteles.hotelid, ciudades.ciudadid,
       TO_CHAR(reservas.FechaInicio, 'MM/YYYY') AS periodo
FROM reservas
INNER JOIN habitaciones ON habitaciones.habitacionid = reservas.habitacionasociada
INNER JOIN hoteles ON hoteles.hotelid = habitaciones.hotelid
INNER JOIN ciudades ON ciudades.ciudadid = hoteles.ciudad
GROUP BY hoteles.hotelid, ciudades.ciudadid,
         TO_CHAR(reservas.FechaInicio, 'MM/YYYY')
ORDER BY MIN(reservas.fechainicio);
```

Versión corregida:

```
SELECT COUNT(reservas.ReservaID) AS num_reservas,
       hoteles.Nombre AS Hotel,
       ciudades.Nombre AS Ciudad,
       TO_CHAR(reservas.FechaInicio, 'MM/YYYY') AS periodo
FROM reservas
INNER JOIN habitaciones ON habitaciones.HabitacionID = reservas.HabitacionAsociada
```

```
INNER JOIN hoteles ON hoteles.HotelID = habitaciones.HotelID
INNER JOIN ciudades ON ciudades.CiudadID = hoteles.Ciudad
WHERE reservas.Estado != 'cancelada'
  AND reservas.FechaInicio BETWEEN :P_FECHA_INICIO AND :P_FECHA_FIN
  AND (:P_HOTEL IS NULL OR hoteles.HotelID = :P_HOTEL)
  AND (:P_CIUDAD IS NULL OR ciudades.CiudadID = :P_CIUDAD)
GROUP BY hoteles.Nombre, ciudades.Nombre,
         TO_CHAR(reservas.FechaInicio, 'MM/YYYY')
ORDER BY MIN(reservas.FechaInicio);
```

Cambios realizados:

1. Se muestran nombres de hotel y ciudad en vez de solo IDs numéricos. 2. Se agregó filtro para excluir reservas canceladas. 3. Se agregó filtro por rango de fechas parametrizado. 4. Se agregaron filtros opcionales por hotel y ciudad.

5.3. RFC3 – Huéspedes por fechas y ciudad

Tablas utilizadas: RESERVAS, HABITACIONES, HOTELES, CIUDADES. Se usa SUM(NumAdultos + NumMenores3) para calcular el total de huéspedes. Los JOINs conectan la reserva con su habitación, hotel y ciudad.

Versión inicial:

```
SELECT hoteles.hotelid,
       SUM(reservas.numadultos+reservas.nummenores3) AS huespedes,
       ciudades.ciudadid
FROM reservas
INNER JOIN habitaciones ON habitaciones.habitacionid = reservas.habitacionasociada
INNER JOIN hoteles ON hoteles.hotelid = habitaciones.hotelid
INNER JOIN ciudades ON ciudades.ciudadid = hoteles.ciudad
WHERE reservas.fechafin >= DATE '2026-04-05'
  AND reservas.fechainicio <= DATE '2026-04-11'
  AND hoteles.hotelid = 3
GROUP BY hoteles.hotelid, ciudades.ciudadid;
```

Versión corregida:

```
SELECT hoteles.Nombre AS Hotel,
       ciudades.Nombre AS Ciudad,
       SUM(reservas.NumAdultos + reservas.NumMenores3) AS Huespedes
FROM reservas
INNER JOIN habitaciones ON habitaciones.HabitacionID = reservas.HabitacionAsociada
INNER JOIN hoteles ON hoteles.HotelID = habitaciones.HotelID
INNER JOIN ciudades ON ciudades.CiudadID = hoteles.Ciudad
```

```
WHERE reservas.Estado != 'cancelada'
  AND reservas.FechaFin >= :P_FECHA_INICIO
  AND reservas.FechaInicio <= :P_FECHA_FIN
  AND (:P_HOTEL IS NULL OR hoteles.HotelID = :P_HOTEL)
  AND ciudades.CiudadID = :P_CIUDAD
GROUP BY hoteles.Nombre, ciudades.Nombre;
```

Cambios realizados:

1. Se muestran nombres en vez de IDs. 2. Se excluyen reservas canceladas. 3. El filtro de hotel se hizo opcional. 4. Se parametrizaron las fechas.

5.4. RFC4 – Ingresos por hotel por rango de fechas

Tablas utilizadas: RESERVAS, HABITACIONES, HOTELES, CIUDADES. Se usa SUM(ValorTotal) para los ingresos por hotel y una window function SUM() OVER (PARTITION BY CiudadID) para el agregado por ciudad, lo que permite ver ambos niveles de agregación en una sola consulta.

Versión inicial:

```
SELECT SUM(reservas.valortotal) AS ingresos_hotel,
       ciudades.ciudadid, hoteles.hotelid,
       SUM(SUM(reservas.ValorTotal))
         OVER (PARTITION BY ciudades.CiudadID) AS ingresos_ciudad
FROM reservas
INNER JOIN habitaciones ON habitaciones.habitacionid = reservas.habitacionasociada
INNER JOIN hoteles ON hoteles.hotelid = habitaciones.hotelid
INNER JOIN ciudades ON ciudades.ciudadid = hoteles.ciudad
WHERE EXTRACT(YEAR FROM reservas.FechaInicio) = 2026
GROUP BY ciudades.ciudadid, hoteles.hotelid;
```

Versión corregida:

```
SELECT hoteles.Nombre AS Hotel,
       ciudades.Nombre AS Ciudad,
       SUM(reservas.ValorTotal) AS Ingresos_Hotel,
       SUM(SUM(reservas.ValorTotal))
         OVER (PARTITION BY ciudades.CiudadID) AS Ingresos_Ciudad
FROM reservas
INNER JOIN habitaciones ON habitaciones.HabitacionID = reservas.HabitacionAsociada
INNER JOIN hoteles ON hoteles.HotelID = habitaciones.HotelID
INNER JOIN ciudades ON ciudades.CiudadID = hoteles.Ciudad
WHERE reservas.Estado != 'cancelada'
  AND reservas.FechaInicio BETWEEN :P_FECHA_INICIO AND :P_FECHA_FIN
```

```
GROUP BY ciudades.CiudadID, ciudades.Nombre, hoteles.Nombre;
```

Cambios realizados:

1. Se muestran nombres en vez de IDs. 2. Se excluyen reservas canceladas. 3. Se cambió el filtro de año fijo (EXTRACT YEAR = 2026) a un rango de fechas parametrizado con BETWEEN.

5.5. RFC5 – Servicios y tipos de habitación más populares

Esta consulta se dividió en dos: RFC5a para tipos de habitación y RFC5b para servicios adicionales.

RFC5a – Tipos de habitación más populares

Tablas utilizadas: HABITACIONES, RESERVAS, HOTELES. Se cuenta el número de reservas por tipo de habitación usando COUNT y se ordena descendentemente.

Versión inicial:

```
SELECT COUNT(reservas.reservaid) AS num_reservas, habitaciones.tipo
FROM habitaciones
INNER JOIN RESERVAS ON RESERVAS.HABITACIONASOCIADA = HABITACIONES.HABITACIONID
INNER JOIN hoteles ON hoteles.hotelid = habitaciones.hotelid
WHERE reservas.fechafin >= DATE '2026-04-05'
  AND reservas.fechainicio <= DATE '2026-04-11'
  AND hoteles.hotelid = 3
GROUP BY habitaciones.tipo
ORDER BY num_reservas DESC;
```

Versión corregida:

```
SELECT habitaciones.Tipo,
       hoteles.Nombre AS Hotel,
       COUNT(reservas.ReservaID) AS Num_Reservas
FROM habitaciones
INNER JOIN reservas ON reservas.HabitacionAsociada = habitaciones.HabitacionID
INNER JOIN hoteles ON hoteles.HotelID = habitaciones.HotelID
WHERE reservas.Estado != 'cancelada'
  AND reservas.FechaFin >= :P_FECHA_INICIO
  AND reservas.FechaInicio <= :P_FECHA_FIN
  AND (:P_HOTEL IS NULL OR hoteles.HotelID = :P_HOTEL)
GROUP BY habitaciones.Tipo, hoteles.Nombre
ORDER BY Num_Reservas DESC;
```

RFC5b – Servicios más populares

Tablas utilizadas: SERVICIOSADICIONALES, SERVICIOSRESERVAS, RESERVAS, HABITACIONES, HOTELES. Se usa un encadenamiento de JOINs desde servicios hasta hotel, contando cuántas reservas usan cada servicio.

Versión inicial:

```
SELECT COUNT(SERVICIOSRESERVAS.RESERVA) AS NUM_RESERVAS,
       SERVICIOSADICIONALES.SERVICIOID, serviciosadicionales.nombre
FROM SERVICIOSADICIONALES
INNER JOIN SERVICIOSRESERVAS ON serviciosreservas.serviciosolicitado
  = serviciosadicionales.servicioid
INNER JOIN RESERVAS ON reservas.reservaid = serviciosreservas.reserva
INNER JOIN habitaciones ON habitaciones.habitacionid = reservas.habitacionasociada
INNER JOIN hoteles ON hoteles.hotelid = habitaciones.hotelid
WHERE reservas.fechafin >= DATE '2026-04-05'
  AND reservas.fechainicio <= DATE '2026-04-11'
  AND hoteles.hotelid = 3
GROUP BY serviciosadicionales.servicioid, Serviciosadicionales.nombre
ORDER BY num_reservas DESC;
```

Versión corregida:

```
SELECT serviciosadicionales.Nombre AS Servicio,
       hoteles.Nombre AS Hotel,
       COUNT(serviciosreservas.Reserva) AS Num_Reservas
FROM serviciosadicionales
INNER JOIN serviciosreservas ON serviciosreservas.ServicioSolicitado
  = serviciosadicionales.ServicioID
INNER JOIN reservas ON reservas.ReservaID = serviciosreservas.Reserva
INNER JOIN habitaciones ON habitaciones.HabitacionID = reservas.HabitacionAsociada
INNER JOIN hoteles ON hoteles.HotelID = habitaciones.HotelID
WHERE reservas.Estado != 'cancelada'
  AND reservas.FechaFin >= :P_FECHA_INICIO
  AND reservas.FechaInicio <= :P_FECHA_FIN
  AND (:P_HOTEL IS NULL OR hoteles.HotelID = :P_HOTEL)
GROUP BY serviciosadicionales.Nombre, hoteles.Nombre
ORDER BY Num_Reservas DESC;
```

Cambios realizados en RFC5a y RFC5b:

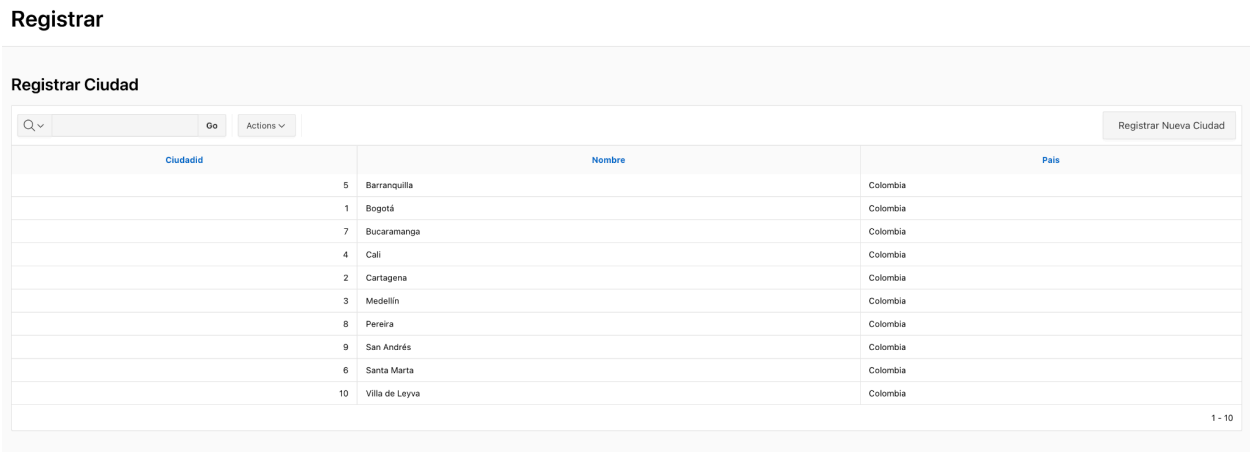
1. Se agregó nombre del hotel como columna visible. 2. Se excluyen reservas canceladas. 3. Se parametrizaron fechas y filtro de hotel. 4. En RFC5b se quitó el ServicioID del SELECT ya que no aporta valor al usuario final.

6. Desarrollo de la aplicación en APEX

La aplicación se desarrolló en Oracle APEX utilizando el enfoque low-code de la plataforma. Se organizaron las funcionalidades en páginas con Interactive Reports, formularios modales (Modal Dialog), Charts y componentes de filtrado. A continuación se documenta cada requerimiento funcional implementado.

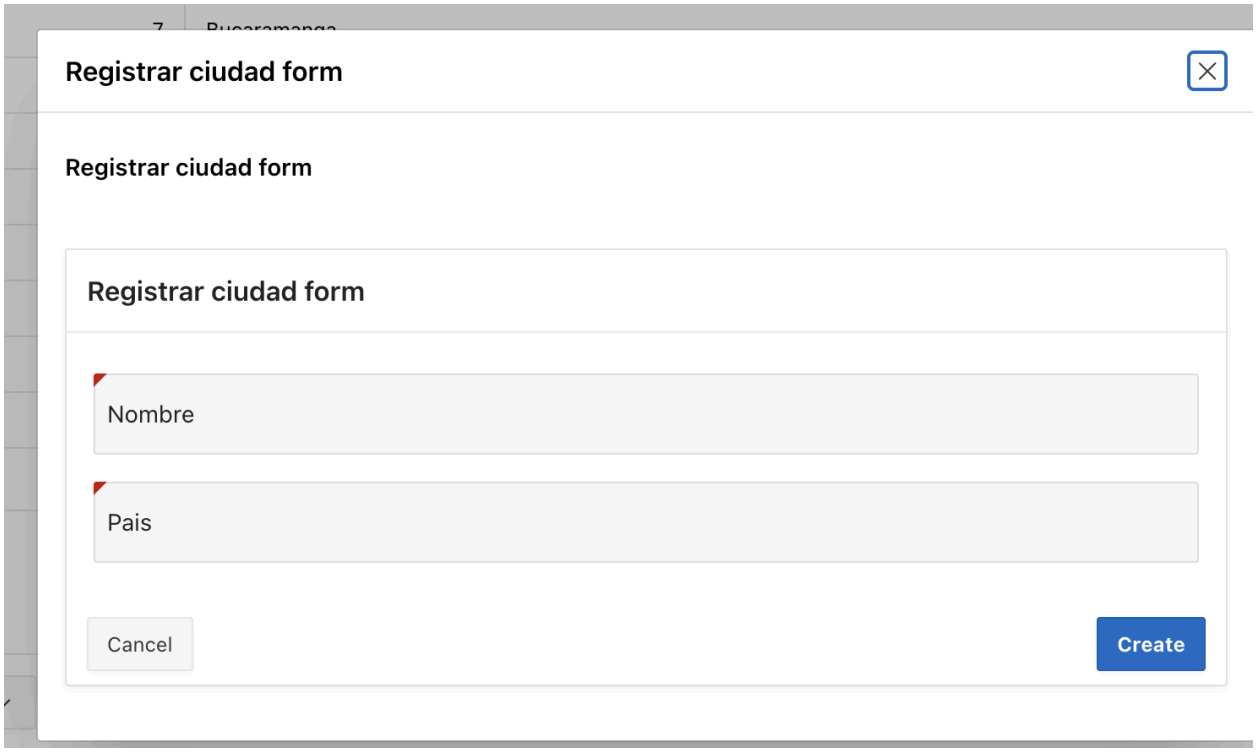
6.1. RF1 – Registrar ciudad

Se implementó como un Interactive Report sobre la tabla CIUDADES en la página "Registrar", que muestra todas las ciudades existentes con columnas de Nombre y País. Se creó un formulario modal (Modal Dialog) en una página aparte que permite insertar nuevas ciudades con los campos Nombre y País. El Interactive Report incluye un botón "Registrar Nueva Ciudad" que abre el formulario modal. El CiudadID se genera automáticamente mediante IDENTITY y se oculta del usuario. El constraint UNIQUE sobre (Nombre, País) impide duplicados.



Ciudadid	Nombre	País
5	Barranquilla	Colombia
1	Bogotá	Colombia
7	Bucaramanga	Colombia
4	Cali	Colombia
2	Cartagena	Colombia
3	Medellín	Colombia
8	Pereira	Colombia
9	San Andrés	Colombia
6	Santa Marta	Colombia
10	Villa de Leyva	Colombia

[CAPTURA: Vista del Interactive Report de ciudades con el botón Registrar Nueva Ciudad]



[CAPTURA: Formulario modal de registro de ciudad en APEX Page Designer]

6.2. RF2 – Registrar hotel

Se implementó con el mismo patrón de Interactive Report + formulario modal, esta vez sobre la tabla HOTELES. El formulario incluye los campos Nombre, Ciudad (como Select List que consulta la tabla CIUDADES mostrando el nombre), Dirección, Teléfono y Descripción General. La FK hacia CIUDADES garantiza que solo se puedan registrar hoteles en ciudades previamente registradas. El Interactive Report muestra todos los hoteles con sus datos y permite editar cualquiera mediante el ícono de lápiz.

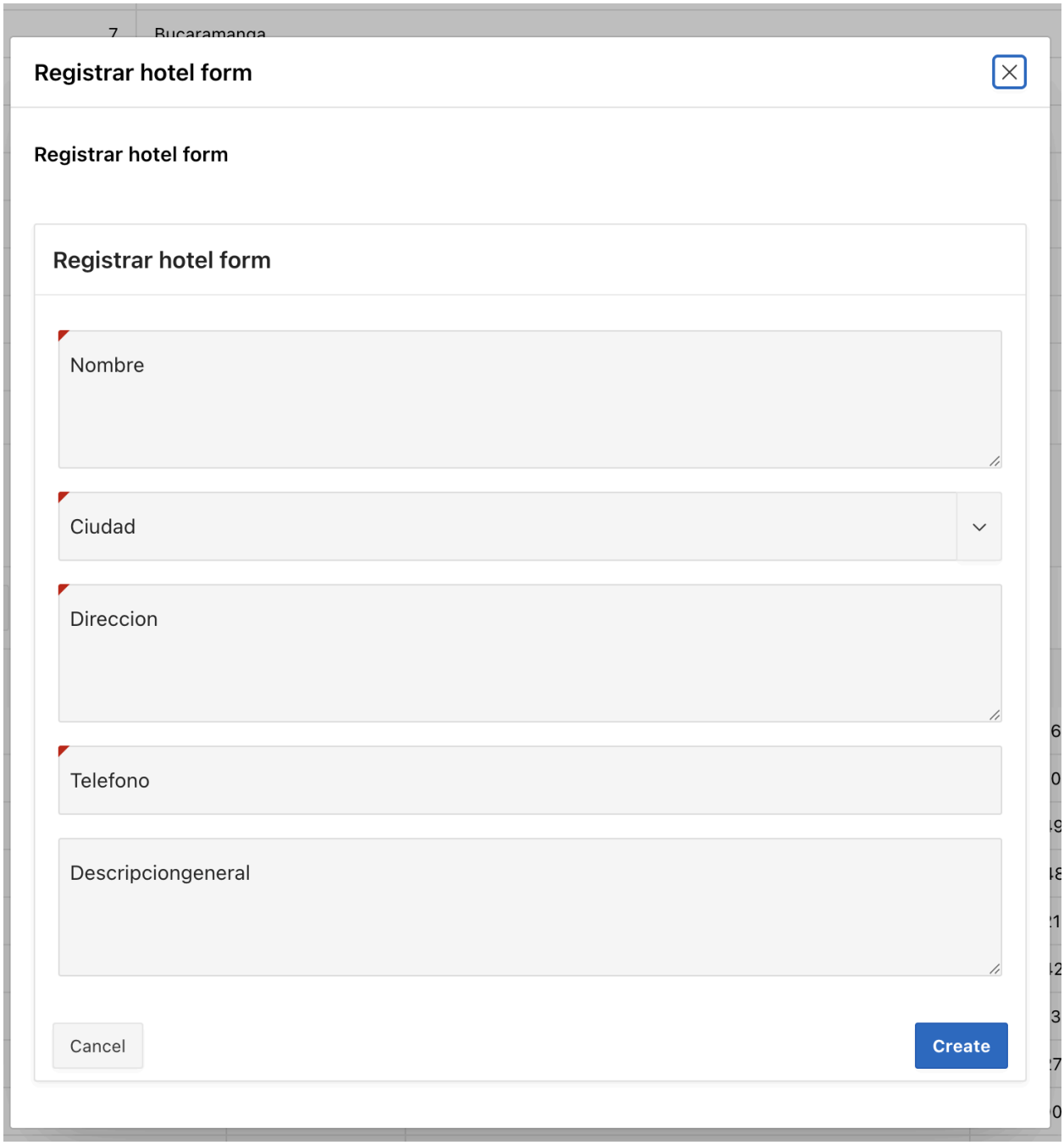
Registrar Hotel

Go | Actions | Registrar hotel

Hotelid	Nombre	Ciudad	Direccion	Telefono	Descripciongeneral
1	Hotel GHL 93	1	Cra 9 #115-06	6013456789	Hotel ejecutivo zona norte Bogotá
2	Hotel Sofitel Legend	1	Calle 10 #2-21	6018950000	Hotel 5 estrellas en La Candelaria
3	Hotel Las Américas	2	Av Las Américas s/n	6056549000	Resort frente al mar en Cartagena
4	Hotel Dann Carlton	3	Cra 43A #7-50	6044448011	Hotel de negocios en El Poblado
5	Hotel Intercontinental	4	Av Colombia #2-72	6028821212	Hotel 5 estrellas frente al río
6	Hotel Dann Cartagena	2	Calle del Arsenal #10-40	6056642066	Hotel colonial en centro histórico
7	Hotel Barranquilla Plaza	5	Cra 51B #79-246	6053613030	Hotel moderno zona empresarial
8	Hotel Zuana Beach	6	Km 4 vía al Rodadero	6054227010	Resort todo incluido frente al mar
9	Hotel Dann Bucaramanga	7	Calle 47 #28-60	6076300300	Hotel céntrico zona comercial
10	Hotel Sonesta Pereira	8	Km 4 vía Cerritos	6063113737	Hotel campestre eje cafetero
11	Hotel Decameron San Andrés	9	Av Colombia #1-19	6085131800	Resort all inclusive isla
12	Hotel Plazuela de San Diego	2	Calle de las Bóvedas #39-29	6056601210	Boutique hotel amurallado
13	Hotel Cosmos 100	1	Calle 100 #21A-41	6016162600	Hotel de convenciones norte Bogotá
14	Hotel Casa Dann Carlton Bogotá	1	Cra 15 #103-60	6014141777	Suite hotel zona financiera
15	Hotel Villa de Leyva Dann	10	Cra 8 #12-31	6087320166	Hotel colonial plaza principal

1 - 15

[CAPTURA: Vista del Interactive Report de hoteles]



[CAPTURA: Formulario modal de registro de hotel mostrando el Select List de ciudades]

6.3. RF2.5 – Registrar servicio

Se implementó también en la página "Registrar" como una tercera región Interactive Report sobre SERVICIOSADICIONALES. El formulario modal incluye los campos Nombre, Descripción (como Textarea), TipoCosto (como Select List con valores estáticos "Por uso" y "Costo Fijo") y Precio (como Number Field). El CHECK constraint CK_SERVAD_TIPO_COSTO valida que el tipo de costo sea uno de los dos valores permitidos.

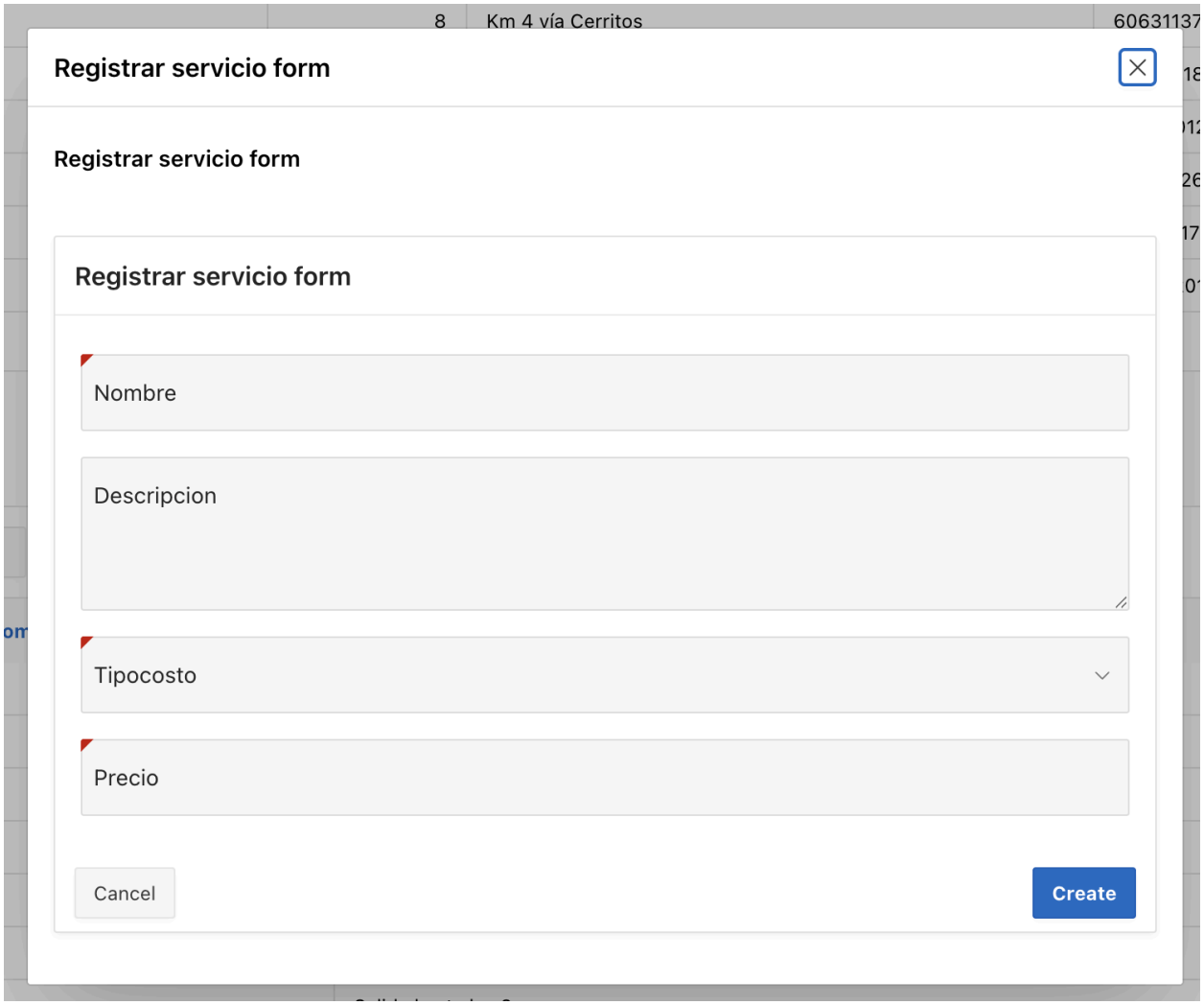
Registrar Servicio

Go | Actions | Registrar Servicio

Servicioid	Nombre	Descripcion	Tipocosto	Precio
1	Desayuno buffet	Desayuno incluido diariamente	Por uso	48000
2	Airport shuttle	Traslado ida y vuelta aeropuerto	Costo Fijo	130000
3	Servicio de masajes	Masaje relajante 60 min	Por uso	190000
4	Check-in temprano	Ingreso desde las 10am	Costo Fijo	55000
5	Valet parking	Servicio de parqueadero con conductor	Por uso	35000
6	Cancelación flexible	Cancelación gratuita hasta 1 día antes	Costo Fijo	25000
7	Check-out tardío	Salida hasta las 3pm	Costo Fijo	45000
8	Cuna para bebé	Cuna adicional en la habitación	Costo Fijo	20000
9	Acceso a sauna	Uso ilimitado de sauna y turco	Costo Fijo	40000
10	Almuerzo buffet	Almuerzo incluido diariamente	Por uso	62000
11	Plan de deportes	Acceso a canchas y actividades deportivas	Por uso	55000

1 - 11

[CAPTURA: Vista del Interactive Report de servicios adicionales]



[CAPTURA: Formulario modal de registro de servicio]

6.4. RF3 – Registrar habitaciones para un hotel

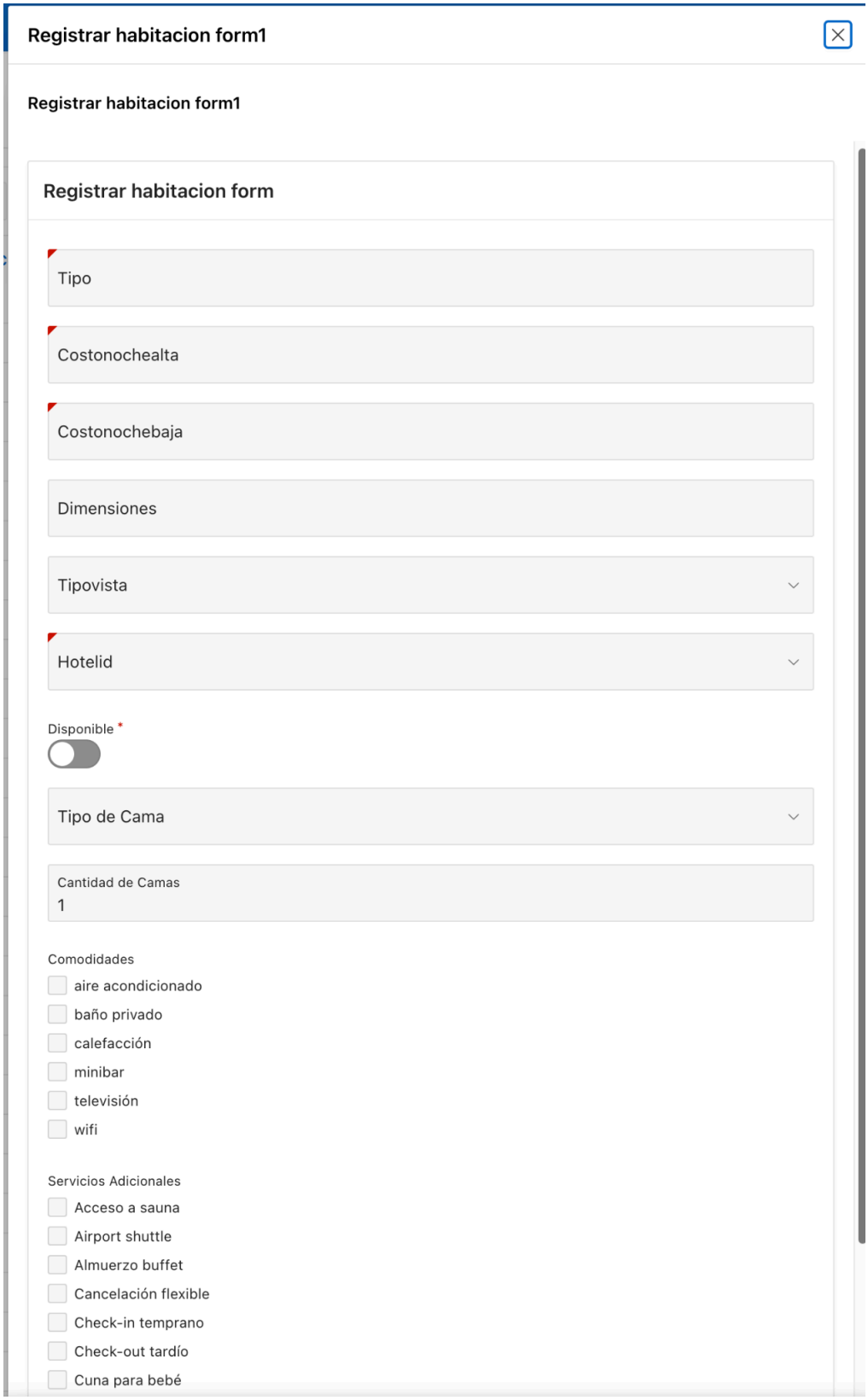
Este es el requerimiento más complejo, ya que involucra múltiples tablas. Se creó una página dedicada "Gestionar Habitaciones" con un Interactive Report sobre HABITACIONES, y un formulario modal que incluye los campos básicos de la habitación (Tipo, CostoNocheAlta, CostoNocheBaja, Dimensiones, TipoVista como Select List, HotelID como Select List, Disponible como Switch) más campos adicionales: Tipo de Cama y Cantidad (para tabla CAMAS), Comodidades (como Checkbox Group con valores estáticos) y Servicios Adicionales (como Checkbox Group con LOV dinámico desde SERVICIOSADICIONALES). Se creó un proceso PL/SQL que, tras insertar la habitación, inserta los registros correspondientes en CAMAS, COMODIDADES y SERVICIOSHOTELTIPO usando APEX_STRING.SPLIT para procesar los valores múltiples de los checkboxes.

Registrar Habitacion

Go Actions Registrar Habitacion

Tipo	Costonochealta	Costonochebaja	Dimensiones	Tipovista	Hotelid	Disponible
Suite	620,000	450,000	48	Ciudad	Hotel GHL 93	1.0
Estándar	195,000	145,000	24	Interior	Hotel GHL 93	1.0
Deluxe	380,000	270,000	35	Ciudad	Hotel GHL 93	1.0
Suite	550,000	390,000	42	Ciudad	Hotel Sofitel Legend	0.0
Estándar	210,000	155,000	26	Interior	Hotel Sofitel Legend	1.0
Deluxe	450,000	320,000	40	Ciudad	Hotel Sofitel Legend	1.0
Deluxe	410,000	295,000	38	Mar	Hotel Las Américas	1.0
Suite	580,000	420,000	45	Mar	Hotel Las Américas	1.0
Estándar	200,000	150,000	22	Mar	Hotel Las Américas	1.0
Familiar	321,266	231,851	42	Ciudad	Hotel Dann Carlton	1.0
Suite Presidencial	994,526	718,683	61	Interior	Hotel Dann Carlton	1.0

[CAPTURA: Formulario modal de registro de habitación con campos de camas, comodidades y servicios]



[CAPTURA: Vista del Interactive Report de habitaciones]

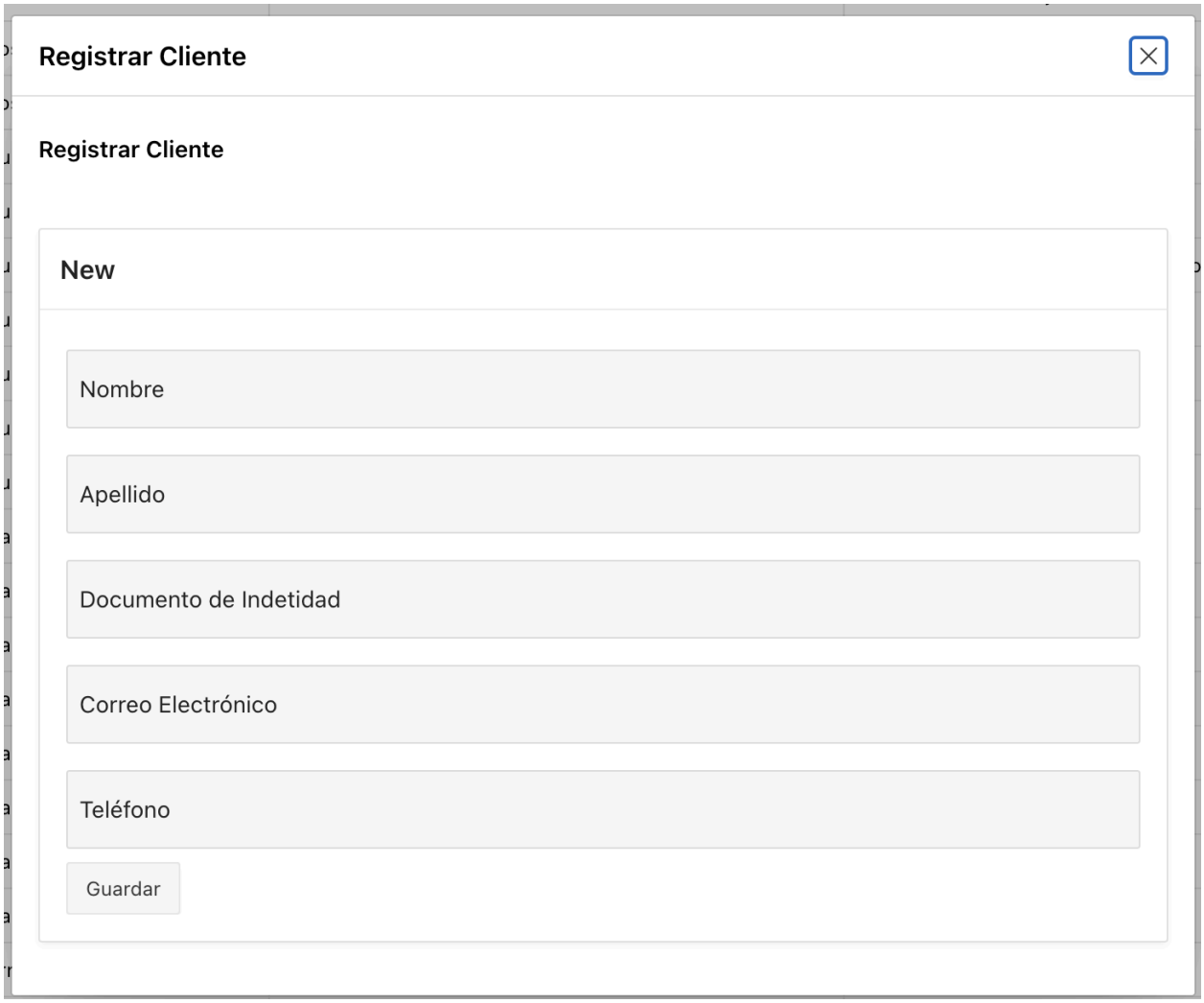
6.5. RF4 – Editar habitaciones

La edición de habitaciones se implementó a través del mismo formulario modal del RF3. Cuando el usuario hace clic en el ícono de lápiz de una habitación existente en el Interactive Report, se abre el formulario con los datos cargados, permitiendo modificar Tipo, costos por noche, dimensiones, tipo de vista y disponibilidad. El proceso DML automático de APEX maneja el UPDATE de la tabla HABITACIONES. Los campos básicos de la habitación son editables directamente a través del formulario.

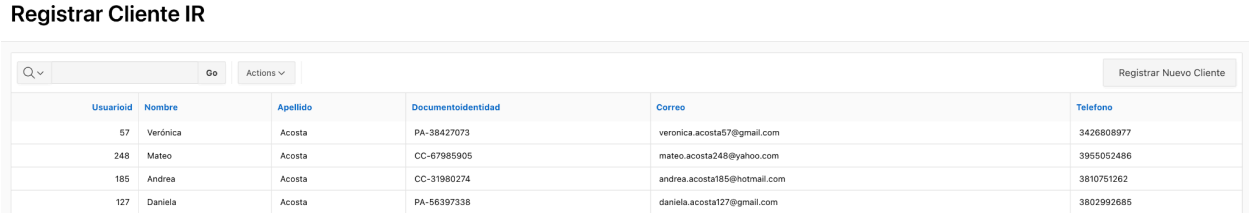
6.6. RF5 – Creación de cuenta de cliente

Se implementó un formulario modal accesible desde dos puntos: un botón en la barra de navegación superior (ícono fa-user-plus junto al nombre de usuario) y desde el Interactive Report de clientes. El formulario contiene los campos Nombre, Apellido, Documento de Identidad, Correo Electrónico y Teléfono. Un proceso PL/SQL inserta primero en USUARIOS y luego en CLIENTES usando RETURNING INTO para obtener el UsuarioID recién generado. El Interactive Report de clientes usa un SQL Query con JOIN entre USUARIOS y CLIENTES para mostrar toda la información consolidada.

[CAPTURA: Botón de crear cuenta en la barra de navegación superior]



[CAPTURA: Formulario modal de registro de cliente]



Usuarioid	Nombre	Apellido	Documentoidentidad	Correo	Telefono
57	Verónica	Acosta	PA-38427073	veronica.acosta57@gmail.com	3426808977
248	Mateo	Acosta	CC-67985905	mateo.acosta248@yahoo.com	3955052486
185	Andrea	Acosta	CC-31980274	andrea.acosta185@hotmail.com	3810751262
127	Daniela	Acosta	PA-56397338	daniela.acosta127@gmail.com	3802992685

[CAPTURA: Interactive Report de clientes]

6.7. RF6 – Gestión de cuenta de cliente

La gestión de cuenta se implementó a través del mismo Interactive Report de clientes. Al hacer clic en el ícono de lápiz de un cliente existente, se abre el formulario modal con los datos

precargados, permitiendo al usuario o al administrador modificar nombre, apellido, correo, teléfono y documento. Este flujo reutiliza la infraestructura del RF5, lo cual mantiene la aplicación simple y consistente.

6.8. RF7 – Realizar reserva de habitación

Se creó una página "Gestionar Reservas" con un Interactive Report que muestra todas las reservas (con JOINs a USUARIOS, HABITACIONES y HOTELES para mostrar nombres en vez de IDs), y un formulario modal para crear nuevas reservas. El formulario incluye: Cliente (Select List con JOIN USUARIOS-CLIENTES), Hotel (Select List), Habitación (Select List cascading dependiente del hotel seleccionado, mostrando solo habitaciones disponibles), Fechas de entrada y salida (Date Pickers), Número de adultos y menores, Servicios adicionales (Checkbox Group cascading dependiente del hotel y tipo de habitación), y Valor total. Un proceso PL/SQL genera un código de confirmación único, inserta la reserva y luego los servicios seleccionados en SERVICIOSRESERVAS. El trigger TRG_RESERVAS_DISPONIBILIDAD valida automáticamente que no haya solapamiento de fechas.

6.9. RF7 – Gestionar reserva por parte del cliente

La gestión de reservas se implementó agregando un botón "Cancelar Reserva" al formulario modal de reservas. Este botón ejecuta un proceso PL/SQL que primero verifica que la fecha de inicio de la reserva sea al menos 3 días en el futuro; si no cumple, lanza un error con RAISE_APPLICATION_ERROR indicando que la cancelación debe hacerse con al menos 3 días de anticipación. Si la validación pasa, actualiza el Estado de la reserva a "cancelada". La modificación de fechas se maneja a través del mismo formulario de edición, donde el trigger de disponibilidad valida automáticamente que las nuevas fechas no se solapen con otras reservas activas.

6.10. Requerimientos funcionales de consulta

Todos los RFCs se implementaron en una página única llamada "Consultas", organizada con un Tabs Container que permite al usuario navegar entre las diferentes consultas mediante pestañas.

RFC1 – Disponibilidad de habitaciones

Se implementó como un Interactive Report dentro de la primera pestaña del Tabs Container. Incluye tres filtros (Hotel como Select List, Fecha de Entrada y Fecha de Salida como Date Pickers) y un botón "Buscar" que ejecuta la consulta parametrizada. El reporte muestra las habitaciones disponibles con su tipo, costos, dimensiones, tipo de vista y hotel.

RFC2 – Reservas a través del tiempo

Se implementó como un Chart de tipo barras en la segunda pestaña. Incluye filtros de Fecha Inicio, Fecha Fin, Hotel (opcional) y Ciudad (opcional). El eje X muestra el período (MM/YYYY) y el eje Y el número de reservas. El gráfico permite identificar visualmente las temporadas con mayor flujo de visitantes.

RFC3 – Huéspedes por ciudad

Se implementó como un Interactive Report en la tercera pestaña. Incluye filtros de Fecha Inicio, Fecha Fin, Ciudad (requerido) y Hotel (opcional). Muestra el total de huéspedes (adultos + menores) agrupado por hotel y ciudad.

RFC4 – Ingresos por hotel

Se implementó como un Chart de tipo barras en la cuarta pestaña. Incluye filtros de Fecha Inicio y Fecha Fin. El eje X muestra cada hotel (con el nombre de la ciudad entre paréntesis) y el eje Y los ingresos totales. Permite visualizar rápidamente qué hoteles generan más ingresos.

RFC5 – Servicios y habitaciones populares

Se implementó como dos Interactive Reports dentro de la quinta pestaña, compartiendo los mismos filtros (Fecha Inicio, Fecha Fin, Hotel opcional). El primer reporte muestra los tipos de habitación más reservados, ordenados descendentemente por número de reservas. El segundo reporte muestra los servicios adicionales más solicitados con el mismo ordenamiento. Ambos permiten identificar las preferencias de los clientes para orientar estrategias de negocio.